A PROJECT REPORT

Submitted by

SUMIT KUMAR (23BCS11219)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

Chandigarh University

APRIL - 2026

CERTIFICATE

This is to certify that the project entitled

"Smart Banking System"

is a bonafide work carried out by

SUMIT KUMAR (23BCS11219)

in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science Engineering from Chandigarh University, Mohali, Punjab during the academic year 2025-2026.

The project work is carried out under the guidance of the department faculty and meets the requirements of the university curriculum.

Project Guide	**Head of Department**
Chandigarh University	Chandigarh University

DECLARATION

I hereby declare that the project work entitled "Smart Banking System" submitted to Chandigarh University, Mohali, Punjab, is a record of an original work done by me under the guidance of the project supervisor. This project work is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science Engineering.

The results embodied in this thesis have not been submitted to any other university or institute for the award of any degree or diploma.

SUMIT KUMAR
Roll No: 23BCS11219
B.E. Computer Science Engineering
Chandigarh University
April 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Chandigarh University for providing me with the opportunity to work on this project. I am deeply thankful to my project guide and faculty members of the Department of Computer Science Engineering for their invaluable guidance, encouragement, and support throughout the development of this project.

I extend my heartfelt thanks to all the faculty members who gave me constructive feedback and suggestions during the project review. I am also grateful to my classmates and friends who supported me at every stage of this project.

Last but not the least, I would like to thank my parents and family for their constant motivation, moral support, and belief in me throughout my academic journey.

SUMIT KUMAR
23BCS11219

ABSTRACT

The Smart Banking System is a full-stack web application developed to simulate the core functionalities of a modern digital banking platform. The system is built using React.js for the frontend, Spring Boot (Java) for the backend REST API, and MySQL as the relational database management system.

The application enables users to register and log in securely using JSON Web Token (JWT) based authentication. Once authenticated, users can open multiple bank accounts (Savings, Checking, or Fixed Deposit), perform deposits and withdrawals, transfer funds between accounts using account numbers, and view a complete transaction history.

The backend is developed with Spring Boot 3.2, Spring Security, and Hibernate ORM. The frontend is built with React 18, using React Router for navigation and Axios for API communication. The application follows a RESTful architecture with role-based access control.

This project demonstrates practical implementation of software engineering concepts including MVC architecture, REST API design, database normalization, JWT authentication, and responsive UI design. The system is intended for educational purposes to demonstrate a complete end-to-end banking solution.

Keywords: Spring Boot, React.js, MySQL, JWT, REST API, Banking System, Full Stack Development

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 8
1.1 Overview 8
1.2 Problem Statement 8
1.3 Objectives 8
1.4 Scope of the Project 8
1.5 Organization of the Report 8
CHAPTER 2: LITERATURE REVIEW 9
2.1 Existing Banking Systems 9
2.2 Technologies Used in Modern Banking 9
2.3 Spring Boot in Enterprise Applications 9
2.4 React.js for Frontend Development 9
2.5 JWT Authentication 9
2.6 Research Gap 9
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN 10
3.1 System Architecture 10
3.2 Technology Stack 10
3.3 Database Design 10
3.4 Entity Relationship Diagram 10
3.5 REST API Design 10
3.6 Security Design 10
3.7 Use Case Diagram Description 11
CHAPTER 4: IMPLEMENTATION 12
4.1 Backend Implementation 12
4.2 Data Models 12
4.3 JWT Authentication Flow 12
4.4 Transaction Management 12
4.5 Frontend Implementation 12
4.6 CORS Configuration 12
CHAPTER 5: TESTING AND RESULTS 14
5.1 Testing Methodology 14
5.2 Test Cases 14
5.3 Results 14
5.4 Performance 14
5.5 Screenshots Description 14
CHAPTER 6: CONCLUSION AND FUTURE SCOPE 15

6.1 Conclusion 15
6.2 Future Scope 15
REFERENCES 16

CHAPTER 1: INTRODUCTION

1.1 Overview

The banking sector has undergone a dramatic transformation with the adoption of digital technologies. Traditional brick-and-mortar banking is rapidly being replaced by internet-based banking systems that offer 24/7 access, real-time transactions, and enhanced security. The Smart Banking System project aims to replicate these capabilities in a simplified yet functional full-stack web application.

1.2 Problem Statement

Traditional banking processes are time-consuming, paper-intensive, and often inaccessible outside business hours. Customers need a digital solution that allows them to manage their accounts, transfer funds, and view transactions from anywhere at any time. This project addresses that need by providing a complete digital banking platform.

1.3 Objectives

The primary objectives of this project are:

- To design and develop a secure, full-stack digital banking application.
- To implement JWT-based user authentication and authorization.
- To enable creation and management of multiple bank account types.
- To support core banking transactions: deposits, withdrawals, and fund transfers.
- To maintain a complete and traceable transaction history for every account.
- To build a responsive and intuitive user interface using React.js.

1.4 Scope of the Project

This project covers the complete development lifecycle of a banking web application, including database design, backend API development, frontend UI development, and integration testing. The scope includes user registration and login, account management, transaction handling, and reporting through a dashboard interface.

1.5 Organization of the Report

The report is organized as follows: Chapter 2 presents a literature review. Chapter 3 describes the system analysis and design. Chapter 4 details the implementation. Chapter 5 covers testing and results. Chapter 6 presents the conclusion and future scope.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Banking Systems

Several digital banking platforms exist today, including net banking portals offered by major banks such as SBI, HDFC, ICICI, and international platforms such as PayPal and Revolut. These systems share common features: user authentication, account management, and fund transfer capabilities. However, most are proprietary, complex, and not suitable for academic study.

2.2 Technologies Used in Modern Banking

Modern banking applications rely heavily on microservices architecture, RESTful APIs, and cloud deployment. Java with Spring Boot has become one of the most widely used frameworks for building enterprise-level backend services due to its robustness, security features, and extensive ecosystem.

2.3 Spring Boot in Enterprise Applications

Spring Boot is a widely adopted framework for building production-ready Java applications with minimal configuration. It integrates seamlessly with Spring Security for authentication, Spring Data JPA for database operations, and Hibernate as the ORM layer. Multiple industry surveys indicate Spring Boot as the top Java framework for backend development.

2.4 React.js for Frontend Development

React.js, developed by Meta, is a component-based JavaScript library for building dynamic user interfaces. Its virtual DOM, unidirectional data flow, and rich ecosystem of libraries make it the most popular frontend framework for single-page applications (SPAs).

2.5 JWT Authentication

JSON Web Token (JWT) is an open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. JWT is widely used in modern web applications as a stateless authentication mechanism, eliminating the need for server-side session storage.

2.6 Research Gap

While much literature exists on individual components (Spring Boot, React, JWT), there are few implementations that combine all three in a complete banking system with proper security, transaction management, and a clean user interface suitable for academic demonstration.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Architecture

The Smart Banking System follows a three-tier architecture consisting of the Presentation Layer (React.js frontend), the Business Logic Layer (Spring Boot backend), and the Data Layer (MySQL database). The frontend communicates with the backend via RESTful HTTP APIs, and the backend communicates with the database using Hibernate ORM through Spring Data JPA.

3.2 Technology Stack

The following technologies were used in the development of this project:

- Frontend: React 18, React Router 6, Axios, CSS3
- Backend: Java 17, Spring Boot 3.2, Spring Security, Spring Data JPA
- Database: MySQL 8.0, MySQL Workbench
- Authentication: JSON Web Token (JWT)
- Build Tools: Maven (backend), npm (frontend)
- IDE: Visual Studio Code

3.3 Database Design

The database consists of three primary tables:

- users - Stores user credentials, personal information, and role.
- accounts - Stores bank account information linked to users.
- transactions - Stores all financial transactions linked to accounts.

Referential integrity is maintained through foreign key constraints. The accounts table references users via user_id, and the transactions table references accounts via account_id.

3.4 Entity Relationship Diagram

The ER diagram shows three entities: User (1) → (Many) Account → (Many) Transaction. Each user can have multiple accounts, and each account can have multiple transactions. The relationship is cascaded such that deleting a user removes all their accounts and transactions.

3.5 REST API Design

The backend exposes the following REST API endpoints:

- POST /api/auth/register — Register a new user
- POST /api/auth/login — Authenticate and receive JWT token
- GET /api/accounts — Get all accounts for the logged-in user
- POST /api/accounts/create — Create a new bank account
- POST /api/transactions/deposit — Deposit money into an account
- POST /api/transactions/withdraw — Withdraw money from an account
- POST /api/transactions/transfer — Transfer funds between accounts
- GET /api/transactions/account/{id} — Get transaction history for an account

3.6 Security Design

All API endpoints except /api/auth/** are protected by JWT authentication. The JwtAuthFilter intercepts every incoming HTTP request, extracts the Bearer token from the Authorization header, validates it, and sets the authentication context. Spring Security ensures that unauthenticated requests are rejected with a 401 Unauthorized response.

3.7 Use Case Diagram Description

The primary actor is the Bank Customer. Key use cases include: Register Account, Login, View Dashboard, Open Bank Account, Deposit Funds, Withdraw Funds, Transfer Funds, and View Transaction History. All use cases except Register and Login require prior authentication.

CHAPTER 4: IMPLEMENTATION

4.1 Backend Implementation

The backend is implemented as a Spring Boot application with a layered architecture. The main layers are:

- Model Layer: Contains JPA entities — User, Account, Transaction — mapped to MySQL tables.
- Repository Layer: Interfaces extending JpaRepository for database operations.
- Service Layer: Business logic for account management and transaction processing.
- Controller Layer: REST controllers exposing API endpoints.
- Config Layer: JWT utility, JWT filter, and Spring Security configuration.

4.2 Data Models

The User entity stores username, email, password (BCrypt hashed), full name, phone, address, and role. The Account entity stores account number, type (SAVINGS/CHECKING/FIXED_DEPOSIT), balance, status, and a foreign key to the user. The Transaction entity stores type, amount, balance after transaction, description, reference number, status, and a foreign key to the account.

4.3 JWT Authentication Flow

When a user logs in, the AuthController authenticates credentials using Spring Security's AuthenticationManager. On success, JwtUtil generates a signed JWT token with the username as subject and a 24-hour expiry. The token is returned to the client. For subsequent requests, the client sends the token in the Authorization: Bearer header. JwtAuthFilter validates the token and sets the SecurityContext.

4.4 Transaction Management

All financial operations (deposit, withdraw, transfer) are wrapped in @Transactional annotations to ensure atomicity. For a transfer, both the debit from the source account and the credit to the destination account occur in the same database transaction. If either operation fails, the entire transaction is rolled back.

4.5 Frontend Implementation

The React frontend is organized into the following modules:

- AuthContext.js: Global state management for user login status and JWT token storage.
- api.js: Centralized Axios instance with JWT interceptor for all API calls.
- LoginPage / RegisterPage: Authentication screens with form validation.
- Dashboard: Summary of account balances and recent transactions.
- AccountsPage: Account listing, creation, deposit, and withdrawal.
- TransactionsPage: Full transaction history with account selector.
- TransferPage: Fund transfer form with balance display.
- Layout: Sidebar navigation with responsive mobile support.

4.6 CORS Configuration

Cross-Origin Resource Sharing (CORS) is configured in SecurityConfig to allow requests from the React frontend running on localhost:3000 and localhost:3001. The configuration allows GET, POST, PUT, DELETE, and OPTIONS methods with all headers.

CHAPTER 5: TESTING AND RESULTS

5.1 Testing Methodology

The project was tested using both manual testing and unit testing approaches. Manual testing was performed through the browser interface, verifying each feature against expected behavior. API endpoints were individually tested using the browser and direct HTTP calls.

5.2 Test Cases

The following test cases were executed:

- TC01: User Registration — Valid inputs → Success message, user saved in DB.
- TC02: User Registration — Duplicate username → Error: 'Username already exists'.
- TC03: User Login — Valid credentials → JWT token returned, dashboard displayed.
- TC04: User Login — Invalid password → Error: 'Invalid username or password'.
- TC05: Open Account — Select Savings, initial deposit ₹5000 → Account created.
- TC06: Deposit — Amount ₹2000 → Balance updated, transaction recorded.
- TC07: Withdraw — Amount exceeding balance → Error: 'Insufficient balance'.
- TC08: Transfer — Valid destination account → Both accounts updated atomically.
- TC09: Transfer — Invalid account number → Error: 'Destination account not found'.
- TC10: Transaction History — Displays all transactions in reverse chronological order.

5.3 Results

All test cases passed successfully. The application correctly handles both valid and invalid inputs. Error messages are displayed in the UI for all error scenarios. Transactions are atomic — no partial updates occur even in failure scenarios.

5.4 Performance

The application demonstrated acceptable performance on a local development environment. The Spring Boot server starts in approximately 26 seconds on first run (subsequent runs are faster due to JVM warm-up). API response times for all endpoints were under 200ms on localhost.

5.5 Screenshots Description

The application interface includes: a clean login page with a split-screen design featuring a dark blue branding panel and a white form panel; a dashboard with four summary cards showing total balance, account count, transaction count, and active accounts; an accounts page showing account cards with deposit/withdraw buttons; and a transfer page with a balance preview card.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The Smart Banking System project successfully demonstrates the development of a complete, functional digital banking application using modern web technologies. The system implements all core banking features — user authentication, account management, deposits, withdrawals, fund transfers, and transaction history — in a secure and user-friendly manner.

The project provided hands-on experience with industry-standard technologies including Spring Boot, Spring Security, JWT, React.js, and MySQL. It reinforced key software engineering principles including RESTful API design, MVC architecture, database normalization, and separation of concerns.

All planned objectives were successfully achieved. The application is fully functional, secure, and ready for demonstration purposes.

6.2 Future Scope

The following enhancements can be made in future versions of the system:

- Mobile Application: Develop a mobile app using React Native for Android and iOS.
- OTP Verification: Add email/SMS OTP verification during registration and login.
- Loan Management: Add loan application, approval, and EMI tracking modules.
- Fixed Deposit Management: Add FD creation, maturity tracking, and interest calculation.
- Admin Dashboard: Build an administrative panel for user management and reports.
- Cloud Deployment: Deploy the application on AWS/GCP/Azure with Docker containers.
- Two-Factor Authentication: Implement 2FA for enhanced account security.
- PDF Statements: Allow users to download monthly account statements as PDF.
- Notifications: Add email and push notifications for transactions.
- Currency Support: Add multi-currency accounts and exchange rate features.

REFERENCES

[1] Spring Boot Official Documentation. (2024). Spring Boot Reference Guide. https://docs.spring.io/spring-boot/docs/current/reference/html/

[2] React Documentation. (2024). React — A JavaScript library for building user interfaces. https://reactjs.org/docs/getting-started.html

[3] MySQL Documentation. (2024). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

[4] JSON Web Tokens. (2024). JWT.io Introduction. https://jwt.io/introduction

[5] Spring Security Documentation. (2024). Spring Security Reference. https://docs.spring.io/spring-security/reference/

[6] Hibernate ORM. (2024). Hibernate ORM 6.x Documentation. https://hibernate.org/orm/documentation/

[7] Walls, C. (2022). Spring in Action, 6th Edition. Manning Publications.

[8] Banks, A., & Porcello, E. (2020). Learning React, 2nd Edition. O'Reilly Media.

[9] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

[10] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. PhD Dissertation, University of California, Irvine.